

9 EXPERIMENTATION DETAILS

Detailed distributions

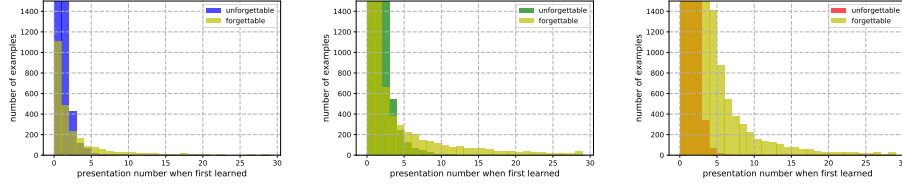


Figure 8: From left to right, distributions of the first presentation at which each unforgettable and forgettable example was learned in *MNIST*, *permutedMNIST* and *CIFAR-10* respectively. Rescaled view where the number of examples have been capped between 0 and 1500 for visualization purposes. Unforgettable examples are generally learnt early during training, thus may be considered as “easy” in the sense of Kumar et al. (2010), i.e. may have a low loss during most of the training.

Misclassification margin

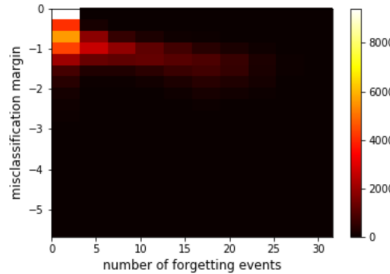


Figure 9: *Left* 2D-histogram of the number of forgetting events and mean misclassification margin across all examples of *CIFAR-10*. There is significant negative correlation (-0.74 , Spearman rank correlation) between mean misclassification margin and the number of forgetting events.

permutedMNIST The *permutedMNIST* data set is obtained by applying a fixed random permutation of the pixels to all the images of the standard *MNIST* data set. This typically makes the data set harder to learn for convolutional neural networks as local patterns, *e.g.* the horizontal bar of the 7, get shuffled. This statement is supported by the two following facts:

- The number of unforgettable examples for *permutedMNIST* is 45181 versus 55012 for *MNIST*.
- The intrinsic data set dimension (Li et al., 2018) of *permutedMNIST* is 1400 compared to 290 for the untouched data set.

Network Architectures We use a variety of different architectures in the main text. Below are their specifications.

The architecture for the *MNIST* and *permutedMNIST* experiments is the following:

1. a first convolutional layer with 5 by 5 filters and 10 feature maps,
2. a second convolutional layer with 5 by 5 filters and 20 feature maps,
3. a fully connected layer with 50 hidden units
4. the output layer, with 10 logits, one for each class.

We apply ReLU nonlinearities to the feature maps and to the hidden layer. The last layer is passed through a softmax to output probabilities for each class of the data set.

The ResNet18 architecture used for *CIFAR-10* is described thoroughly in DeVries & Taylor (2017), its implementation can be found at <https://github.com/uoguelph-mlrg/Cutout>.

The second one is a WideResNet (Zagoruyko & Komodakis, 2016), with a depth of 28 and a widen factor of 10. We used the implementation found at <https://github.com/meliketoy/wide-resnet.pytorch>.

The convolutional architecture used in Section 6 is the following:

1. a first convolutional layer with 5 by 5 filters and 6 feature maps,
2. a 2 by 2 max pooling layer
3. a second convolutional layer with 5 by 5 filters and 16 feature maps,
4. a first fully connected layer with 120 hidden units
5. a second fully connected layer with 84 hidden units
6. the output layer, with 10 logits, one for each class.

Optimization

The *MNIST* networks are trained to minimize the cross-entropy loss using stochastic gradient descent with a learning rate of 0.01 and a momentum of 0.5.

The ResNet18 is trained using cutout, data augmentation and stochastic gradient descent with a 0.9 Nesterov momentum and a learning rate starting at 0.1 and divided by 5 at epochs 60, 120 and 160.

The WideResNet is trained using Adam (Kingma & Ba, 2014) and a learning rate of 0.001.

10 STABILITY OF THE FORGETTING EVENTS

In Fig 10, we plot precision-recall diagrams for the unforgettable and most forgotten examples of *CIFAR-10* obtained on ResNet18 after 200 epochs and various prior time steps. We see in particular that at 75 epochs, the examples on both side of the spectrum can be retrieved with very high precision and recall.

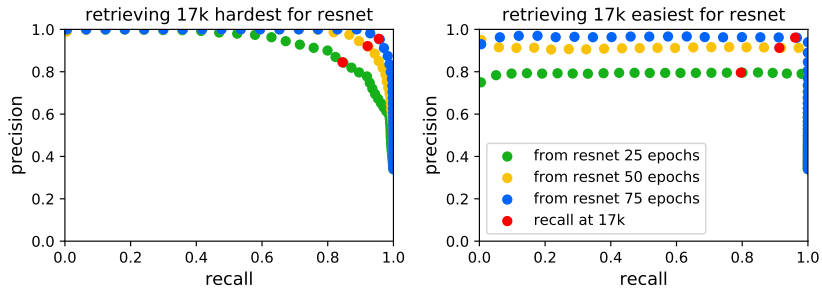


Figure 10: Right: precision and recall of retrieving the unforgettable examples from a full run of ResNet18 (200 epochs), using the example ordering after 25, 50, and 75 epochs. The unforgettable examples are retrieved with high precision and recall after 50 epochs. Left: same plot for the 17k examples with the most forgetting events.

11 Noising THE DATA SETS

In Section 4, we analyzed the effect of adding *label noise* on the distribution of forgetting events. Here, we examine the effect of adding *pixel noise*, i.e. noising the input distribution. We choose to corrupt the inputs with additive Gaussian noise with zero mean and we choose for its standard deviation to be a multiple of channel-wise data standard deviation (i.e., $\sigma_{\text{noise}} = \lambda \sigma_{\text{data}}$, $\lambda \in \{0.5, 1, 2, 10\}$). Note that we add the noise after applying a channel-wise standard normalization

step of the training images, therefore $\sigma_{\text{data}} = 1$ (each channel has zero mean, unit variance, this is a standard pre-processing step and has been applied throughout all the experiments in this paper).

The forgetting distributions obtained by noising all the dataset examples with increasing noise standard deviation are presented in Figure 11. We observe that adding increasing amount of noise decreases the amount of unforgettable examples and increases the amount of examples in the second mode of the forgetting distribution.

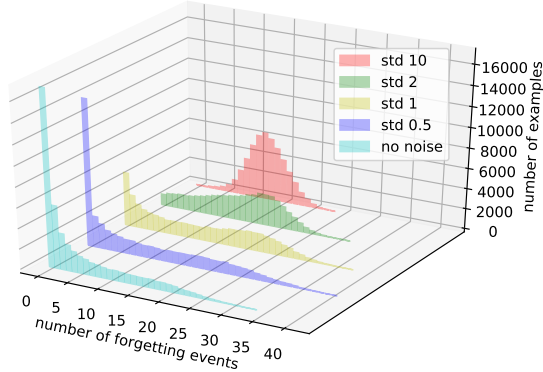


Figure 11: Distribution of forgetting events across all training examples in *CIFAR-10* when all training images are augmented with increasing additive Gaussian noise. The presence of increasing amount of noise decreases the amount of unforgettable examples and increases the amount of examples in the second mode of the forgetting distribution.

We follow the noisy-labels experiments of Section 4 and we apply the aforementioned pixel noise to 20% of the training data ($\sigma_{\text{noise}} = 10$). We present the results of comparing the forgetting distribution of the 20% of examples before and after noise was added to the pixels in Figure 12 (Left). For ease of comparison, we report the same results in the case of label noise in Figure 12 (Right). We observe that the forgetting distribution under pixel noise resembles the one under label noise.

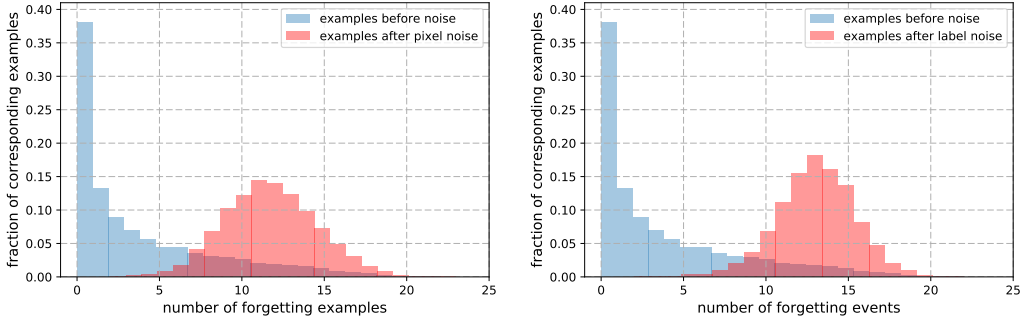


Figure 12: Distribution of forgetting events across all training examples in *CIFAR-10* when random 20% of training examples undergo *pixel noise* ($\sigma_{\text{noise}} = 10$) (Left) or *label noise* (Right) (same as Figure 3). We observe that the forgetting distribution under pixel noise resembles the one under label noise.